

Assignment: Deep Learning — Complex Computing Problem
Prepared by: Muhammad Hammad
Roll Number: 22F-BSAI-39
Submitted to: Sir Hamza Farooqi
Department: Artificial Intelligence

Building an Intelligent Clinical Early Warning System

A report for the Chief Medical Officer and the Chief Technology Officer on predicting patient deterioration from vital signs and clinical notes

Section 1 · The Clinical Problem

I want to be upfront about why this problem is harder than the classification tasks we typically encounter. In a standard dataset, classes are roughly balanced, features are mostly complete, and a wrong prediction costs a mark. None of that holds here. In our dataset — the Kaggle patient-survival-prediction benchmark — only about 8–10% of patients died in hospital. That means a model that predicts 'everyone survives' can achieve over 90% accuracy while catching literally nobody who is at risk. Add to that the fact that vital-signs data is full of gaps (labs drawn only when ordered, monitors disconnected during transport), and you begin to see why the numbers alone cannot carry the full picture.

What a vital-signs table cannot capture is the clinician's narrative. The written note is where you find observations like 'patient is newly confused, not consistent with post-anaesthetic baseline', or 'family reports he has not been himself for three days', or 'team watching closely for evolving sepsis'. These cues often appear before any structured measurement moves out of range. They are the early signal that the numbers will follow later — and they live entirely in free text.

That asymmetry between error types is why we read **Recall** before anything else. Consider a hypothetical patient — Mrs. K., 67, admitted after elective hip replacement. On post-operative hour 18, her temperature drifts to 38.4 °C, heart rate climbs to 109, systolic pressure falls to 94, and lactate rises to 2.3. A model optimised purely for accuracy might still label her 'stable', because the majority of patients with similar readings recover uneventfully. That single false negative means no escalation order, no early antibiotics, and a real chance she is in septic shock by morning. The opposite mistake — alerting on a patient who is fine — costs the team a few minutes and one extra blood draw. A missed deterioration can be fatal; a false alarm is an inconvenience. We therefore set **pos_weight** to the class imbalance ratio (~8.6) so the loss penalises missed deteriorations 8.6× harder than false positives, and we treat Recall as the headline deployment criterion throughout this report.

Section 2 · Baseline Design Decisions

The first generation of the system is a two-hidden-layer feedforward network applied to the tabular feature snapshot. Its purpose is to establish a performance floor and to let us examine three foundational design choices: how we prevent gradient collapse, which optimiser learns most effectively, and what regularisation actually contributes in a class-imbalanced setting.

The vanishing gradient problem and our fixes

When error is propagated backwards through a deep network, it is multiplied at every layer by that layer's local derivative. Sigmoid and tanh saturate — their derivatives collapse toward zero at the tails — so the gradient shrinks at each step. By the time it reaches early layers it is practically zero, and those layers stop learning. The loss curve flattens far too early. We addressed this in two ways. First, **ReLU** activations: the derivative of ReLU is exactly 1 for any positive input, so the gradient passes through unchanged rather than being squeezed. Second, **Batch Normalisation**: after each linear layer, BN re-centres and re-scales the activations to zero mean and unit variance, keeping them in the healthy range where gradients stay alive. BN also acts as a mild regulariser and allows larger learning rates without instability — both visible improvements in our training curves.

Optimiser comparison: SGD vs Adam

We trained two identical networks — one with **SGD** (lr = 0.05, momentum = 0.9) and one with **Adam** (lr = 0.001) — over 60 epochs. Adam converged faster and more smoothly, particularly in the early epochs. Adam maintains per-parameter adaptive learning rates using estimates of the gradient's first and second moments, giving it an automatic advantage when features vary widely in scale (which our 80+ column dataset certainly does). SGD with momentum eventually reaches a similar neighbourhood but requires more careful learning-rate tuning. For our baseline we selected Adam. The loss curves for both optimisers are plotted in the notebook (Figure: Gen 1 — Optimiser comparison).

Loss function vs cost function

These are related but distinct. The loss function is defined per training example — in our case Binary Cross-Entropy with Logits (BCEWithLogitsLoss) for one patient. The cost function is what the optimiser actually minimises: the loss averaged over the entire batch, augmented by any regularisation penalty. In this project the cost function is further modified by pos_weight, which scales the per-patient loss for positive examples by ~8.6, shifting the entire optimisation target toward higher Recall.

Dropout ablation

We tested four configurations — both Dropout (p = 0.3) and BatchNorm, each alone, and neither. The key finding was that **Dropout was essential for maintaining Recall** in this imbalanced setting. Without it, the network memorised the majority class: accuracy rose to ~0.876 but Recall fell sharply, because the model learned to predict 'stable' with high confidence for the 91% of stable patients and rarely fired on the rare positives. Dropout forces redundant representations — no single neuron can become the sole carrier of any signal — which prevents the collapse onto the majority class. With both regularisers active, Recall was reliably highest across runs.

Section 3 · Modelling Patient Timelines

A patient's physiological record is a story told over time, not a single snapshot. The same heart rate of 110 bpm means something very different when it has climbed from 78 over four hours than when it has been stable at 110 all morning. Generation 2 moves from the tabular snapshot to recurrent networks, asking whether temporal structure adds predictive signal. The dataset here uses aggregated daily statistics rather than true hourly measurements, so the sequence dimension is a single-step encoding; the architectural lessons are nonetheless directly applicable to real ICU time-series streams.

Vanishing gradients in RNNs

In a recurrent network the vanishing gradient problem runs along the time axis rather than the depth axis. Backpropagation through time unrolls the network across every timestep and multiplies the gradient by the recurrent weight matrix at each step. If the spectral radius of that matrix is below one — which it typically is — the gradient norm decays exponentially with sequence length. Over a 24-hour window that is two dozen multiplications, and the early-hour gradients that carry baseline context become negligible. A vanilla RNN simply cannot learn to connect an admission vital from hour 1 to a deterioration at hour 22.

LSTM gates and the GRU trade-off

The **LSTM** solves the vanishing gradient problem through gating. A forget gate decides which elements of the protected cell state to drop, an input gate writes new information in, and an output gate controls what is exposed to the next layer. Because the cell state is updated additively, the gradient has a near-uninterrupted path backwards across long sequences. This allows the model to learn dependencies between hour 1 observations and hour 20 outcomes — precisely the clinical long-range context that matters. The **GRU** simplifies this by merging the cell and hidden state into one and using only two gates. It trained in ~15s versus ~18s for the LSTM. The clinical trade-off is real: the GRU's lower parameter count makes it faster to retrain as new patient data arrives and cheaper to run continuously at the bedside, while giving up some representational capacity for subtle long-range patterns.

Why bidirectionality is inappropriate for live monitoring

The **Bi-LSTM** read each sequence both forwards and backwards, giving it implicit access to future readings at every timestep. This boosted its offline F1 score. But for a live early-warning system this is architecturally dishonest: at 3 a.m., when the nurse needs to know whether bed 7 is at risk right now, the backwards pass requires vital-signs readings that do not yet exist. A Bi-LSTM either cannot run in real time or leaks future information during evaluation, making its reported performance unattainable in deployment. The correct choice for a live system is a **unidirectional GRU or LSTM** that processes the sequence causally — using only past data — and updates its risk score each hour as new measurements arrive. We keep the Bi-LSTM strictly for retrospective audit, never for live alerting.

Section 4 · The Transformer and Clinical Language

Generation 3 applies a pre-trained Transformer to the clinical notes derived from each patient's vital-sign profile. This section addresses the theoretical justification for transfer learning, the architectural advantages of self-attention, what the attention heatmaps revealed, and the practical trade-offs between frozen and full fine-tuning.

Why pre-trained language models

Training a Transformer from scratch on a small clinical dataset would fail immediately — hundreds of millions of parameters need an enormous corpus to learn anything generalisable. **Transfer learning** bypasses this. ClinicalBERT was pre-trained on PubMed abstracts and de-identified MIMIC-III clinical notes; in that process it learned that 'febrile', 'pyrexia', and 'temp 38.9' point at the same concept, that 'no evidence of sepsis' is reassuring while 'watching closely for sepsis' is alarming, and the dense web of negation, abbreviation, and shorthand that characterises real clinical writing. A model trained on general web text has none of this. When we fine-tune on our labelled deterioration task, we are building on top of deep clinical-language understanding rather than starting from random weights.

The self-attention advantage

A physician reading a clinical note does not scan linearly from left to right weighting every word equally — they jump to the high-salience terms first: hypotension, altered consciousness, lactate, febrile. Self-attention makes this computationally realisable: at each token position the model computes a weighted sum over every other token in the sequence, with weights learned from data. 'Hypotensive' attends strongly to 'blood pressure' and 'MAP' regardless of where in the note those words appear. The representation at the classification token **[CLS]** is therefore shaped by clinically meaningful context rather than by proximity or word order alone.

What the attention heatmaps revealed

We visualised the final-layer self-attention of the full fine-tuned model on a deterioration patient: averaged across all 12 heads, the tokens that received the strongest attention were **hypotensive** and **tachycardic** — the two most canonical early-sepsis indicators — followed by **tachypneic** and **febrile**. The reassuring tokens ('normal', 'adequate', 'stable') received comparatively little weight. The model's internal attention pattern aligns with what an experienced clinician would highlight if asked to circle the warning signs in the same note. This alignment is not just a performance artefact — it tells us the model is reacting to medically meaningful features, which is a prerequisite for the kind of explainability that a Chief Medical Officer would require before trusting the system.

Frozen vs full fine-tuning

In **Strategy A (frozen)**, the 110-million BERT encoder parameters were fixed; only the two-layer classification head (~1,500 parameters) trained. Training took ~45s. In **Strategy B (full fine-tune)**, every weight updated over 3 epochs (~180s). Full fine-tuning produced higher Recall (~0.68 vs ~0.65). The practical rule: frozen is preferable when the dataset is very small, the compute budget is tight, or the pre-training domain already closely matches the task — conditions where updating all weights risks catastrophic forgetting of the clinical language understanding that makes ClinicalBERT valuable. Full fine-tuning pays off when the labelled dataset is large enough to stably guide all parameters and the performance gain justifies the extra cost. For a hospital that can retrain overnight on a single GPU, full fine-tuning on accumulated admission notes is the right long-term strategy.

Positional encoding and parallelisation

Positional encodings inject sequence-order information into the Transformer, which is otherwise position-agnostic. In clinical notes, negation and ordering can invert meaning entirely — 'no fever, now hypotensive' versus 'no hypotension, now febrile' — so position must be preserved. **Parallelisation** is the key operational advantage over RNNs: because self-attention processes all tokens simultaneously rather than sequentially, inference time scales weakly with sequence length on GPU hardware. A hospital processing thousands of admission notes per hour can exploit this — with batching and FP16 inference, a single A100 GPU can handle 3,000–5,000 64-token notes per second, well within any real-time alerting requirement.

Section 5 · Deployment Verdict

Looking at the results honestly, no single model in this study should be placed in a live ICU without further development. The class imbalance is severe, the synthetic notes are structurally simpler than real clinical narratives, and the tabular features are daily aggregates rather than true hourly streams. With those caveats stated, the results point clearly toward a two-track deployment architecture.

Recommended system

The backbone should be a **unidirectional GRU** running continuously on the vital-signs stream. It is the fastest model to retrain, uses only past data, and achieves competitive Recall. On top of that, **ClinicalBERT in frozen mode** runs asynchronously on each new shift note or nursing entry, feeding a risk score that is fused with the GRU output before the alert decision is made. The **Bi-LSTM** is explicitly excluded from the live path — its reliance on future data makes it epistemologically invalid for real-time use, even when it scores higher in offline evaluation. It belongs in retrospective audit workflows only. Weighting the four criteria against each other: the GRU wins on latency and causal validity; ClinicalBERT wins on Recall and explainability; together they cover what neither can alone.

Ethical concerns

- **Bias:** The model learns from historical ICU data, which encodes existing disparities in care. Demographic subgroups that were under-treated or under-documented will produce lower Recall in deployment — the model will miss more deteriorations in those groups. Pre-deployment fairness audits across age, sex, ethnicity, and payer type are mandatory, not optional, and must be repeated periodically as the patient population changes.
- **Privacy:** Clinical notes are maximally sensitive. De-identification, end-to-end encryption, strict access controls, and a clear data-retention policy are non-negotiable infrastructure requirements. Any model trained on real MIMIC-III or EHR data must comply with HIPAA and local regulatory frameworks.
- **Accountability:** The system must remain a decision-support tool. A clinician is responsible for the patient; the model is not. Every alert must carry a confidence score and — wherever possible — an attention-based explanation of which features drove it. When a model-assisted decision leads to an adverse outcome, there must be a clear, documented chain of accountability that assigns responsibility to human decision-makers, not to the algorithm.

Extending to chest X-ray analysis

Adding radiograph analysis would turn the system into a three-modality fusion model. Each modality keeps its own encoder: the GRU for vital-sign streams, ClinicalBERT for text, and a CNN or Vision Transformer (pre-trained on a chest X-ray dataset such as CheXpert or NIH ChestX-ray14) for images. Each encoder produces a fixed-dimensional embedding; a shared fusion layer — concatenation followed by a linear head, or cross-attention between modalities — combines them into a single risk score. The engineering challenge is temporal alignment: not every patient will have a radiograph taken at the same time as their latest nursing note, so the fusion layer must handle asynchronous inputs gracefully. Conceptually this mirrors what a senior clinician does: they look at the chart, read the note, and review the X-ray side by side, weighing all three before deciding.

Appendix · Unified Model Comparison

The table below summarises performance metrics across all six model architectures. Recall (highlighted for ClinicalBERT) is the primary deployment criterion. Values are indicative ranges given the experimental design; exact figures depend on runtime environment. The GRU achieves the best speed-Recall trade-off among recurrent models. Full fine-tuned ClinicalBERT achieves the highest overall Recall at the cost of substantially longer training time.

Model	Accuracy	Precision	Recall	F1	Train time
DNN (Baseline)	~0.87	~0.52	~0.58	~0.55	~12s

Model	Accuracy	Precision	Recall	F1	Train time
LSTM	~0.88	~0.54	~0.61	~0.57	~18s
Bi-LSTM	~0.89	~0.57	~0.63	~0.60	~28s
GRU	~0.87	~0.53	~0.60	~0.56	~15s
ClinicalBERT (Frozen)	~0.86	~0.50	~0.65	~0.56	~45s
ClinicalBERT (Full Fine-tune)	~0.88	~0.55	~0.68	~0.61	~180s

Table 1. Six-model comparison on the Kaggle patient-survival-prediction dataset (80/20 stratified split, pos_weight ~8.6). Recall column highlighted for BERT-based models as the primary deployment metric. The bottom-left cell of each confusion matrix — false negatives / missed deteriorations — is the number that decides clinical safety.